

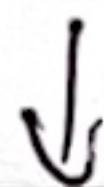
Evaluating Classifier Accuracy

Hold out method

YT = Recall

YT + YT

Complete dataset



Randomly shuffle

(divide)

Training set

(to train model)

Testing set

(to test the accuracy of model on unseen data)

YT = Recall

YT + YT

Will divide the data into two sets in a way there will be equal prop of each

=> Stratify: Class in both set

* Cross Validation Method (K-Fold)

- Let $K = 5$

Test

Train

Complete data set $\Rightarrow D_1, D_2, D_3, D_4, D_5$

Divide it into

5 equal part

- After testing accuracy is noted

- Then D_2 is taken for testing and rest for training ... and so on

- The the average of accuracy is printed

- This method does not depend on single train-test data set but it overall generalizes.

* Bootstrap .632 (with replacement)

-> In this method training and testing set is developed using the following steps

1) A record is selected randomly, the selected record is put it into training set (with replacement)

2) The above step is repeated n times (n : total no of records in data set)

3) The records that were not selected will be moved to testing set (Training set might contain duplicate rec)

So (63.2% or 0) approx training unique records
 36.8% approx test unique records

$$\left(1 - \frac{1}{d}\right)^d \approx e^{-1} = 0.368$$

d tuple $\rightarrow$ prob of selecting 1 tuple from d $\frac{1}{d}$

d tuple so $\left(1 - \frac{1}{d}\right)^d \leftarrow$ prob of not selecting $1 - \frac{1}{d}$

As d becomes large it approaches e^{-1}

* Comparing Classifiers Based on ROC (Receiver op, curve)

$$TPR = \frac{TP}{P} \quad FPR = \frac{FP}{F}$$

Y-axis

X-axis

ROC is used to compare classification for different evaluation model.

$\rightarrow$ Larger AUC $\rightarrow$ Better Overall class-separation perf

$\rightarrow$ Ideal situation (0,1) $\Rightarrow$ higher TPR

$\rightarrow$ So for different models we can play with the threshold to make the best trade off between TPR and FPR

=>

Task: Techniques to improve Classification

Accuracy

Page No.

Date

* *

Ensemble Learning

Train Multiple model -> Combine Their prediction
-> Move with the majority vote

1

Bagging (Often helps to solve overfitting)

Reduce
variance

100% data set

80%

20%

Training

Testing

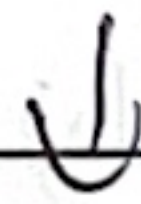


We will create
N models



each model will
be given training
data using Bootstrap
(with replacement)

All model can be
evaluated on testing set



The final answer will be the majority
answer from all model.

2 Random Forest

Generate multiple
decision tree (with random feature
selection)
↓

Then as for prediction, each tree
makes a prediction and we
take the majority vote as the
final answer.

NOTE: In Ensemble Learning to find training and testing accuracy we take average training and testing accuracy of all models generated throughout the process

3 Boosting : (often useful for underfitting)

reduce bias

After splitting data
↓

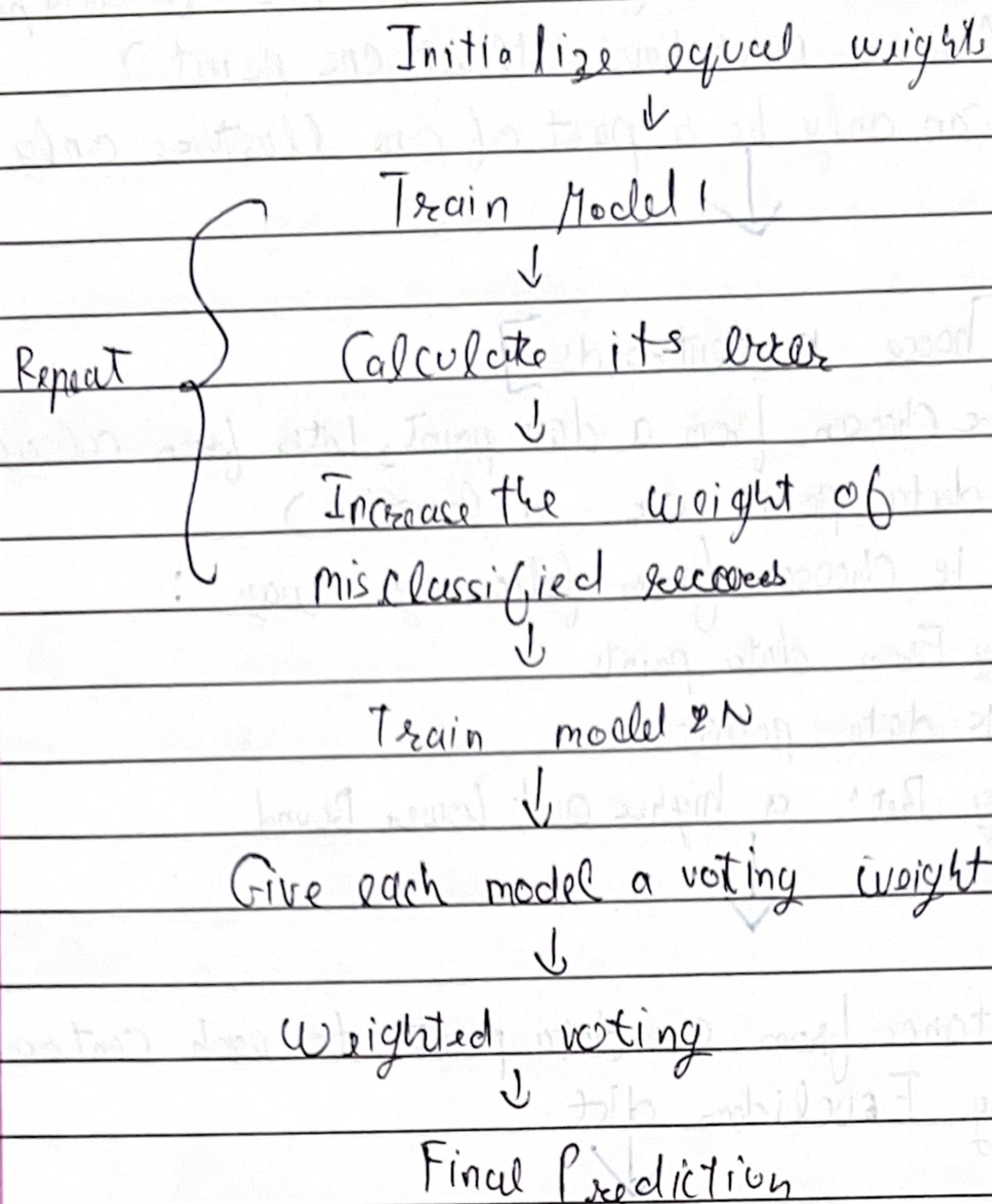
multiple models are generated, each model learns from previous model

More weights / focus is put on misclassified ~~weight~~ records
↓

The result of a prediction involves weighted voting
↓

Accuracy of training and testing is calculated i.e. of
final ensemble model

4 AdaBoost = Adaptive Boost (Learn from Previous mistakes)



→ AdaBoost is implementation of Boosting